

CE 206: Engineering Computation Sessional

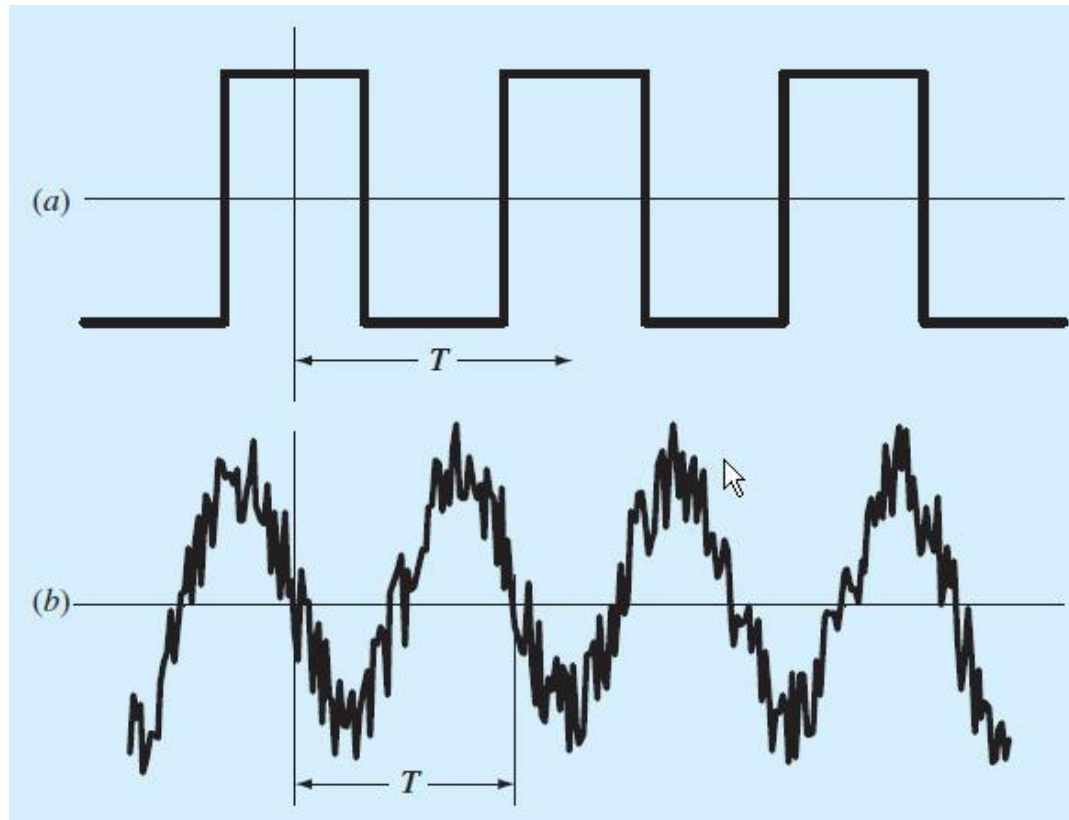
Fourier Analysis

Periodic Functions

A periodic function is one for which

$$f(t) = f(t + T)$$

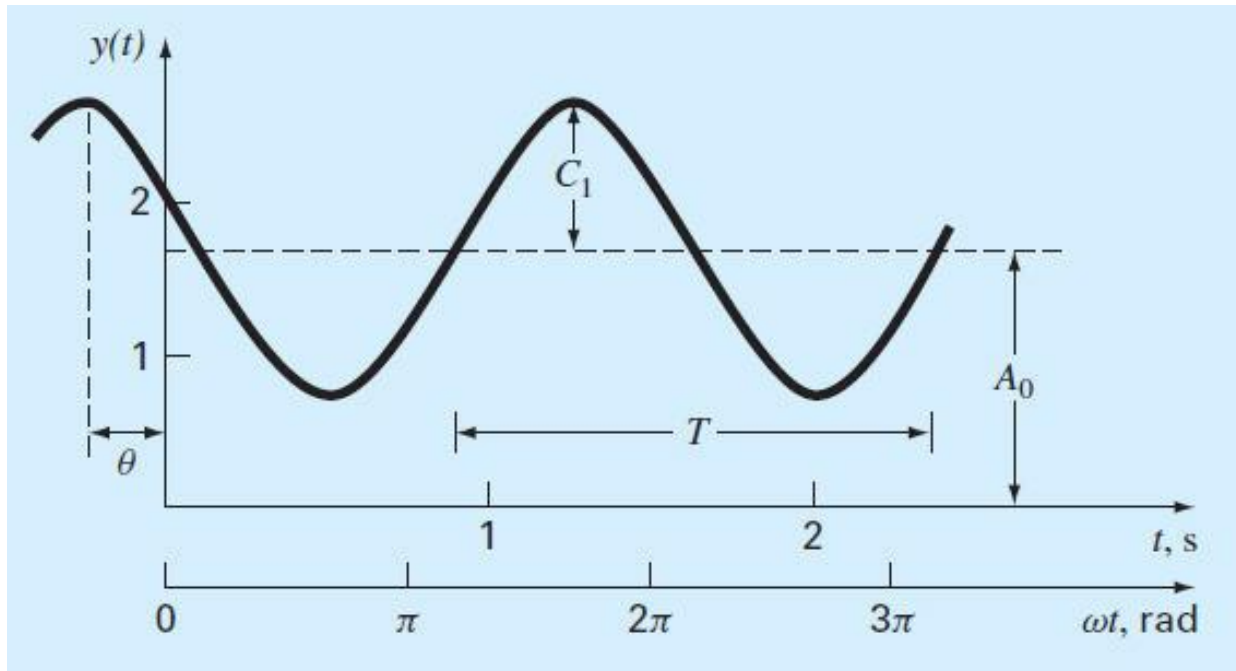
where T = the period



Sinusoids

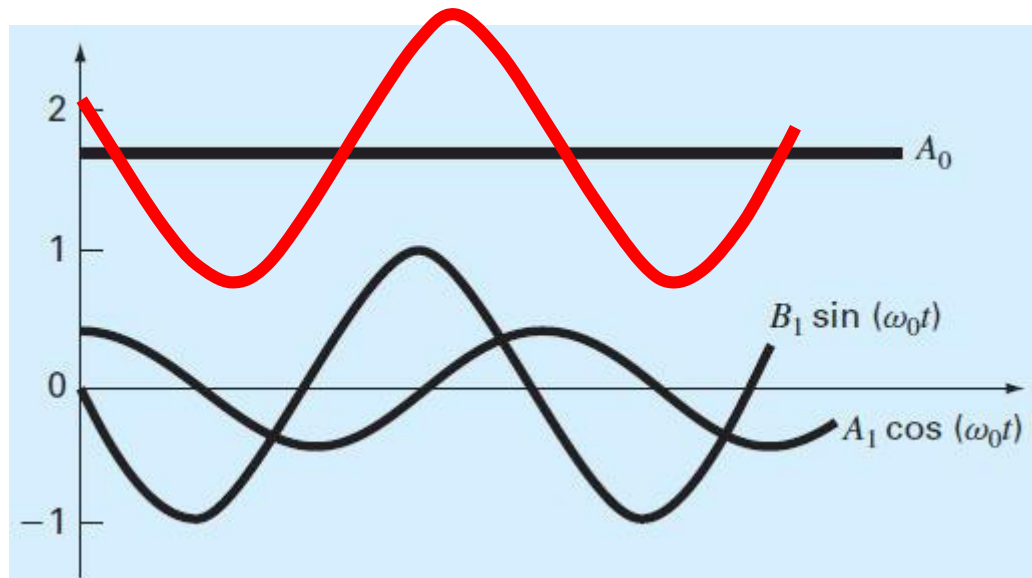
$$f(t) = A_0 + C_1 \cos(\omega_0 t + \theta)$$

Mean value Amplitude Angular frequency Phase shift



Alternative Expressions of Sinusoids

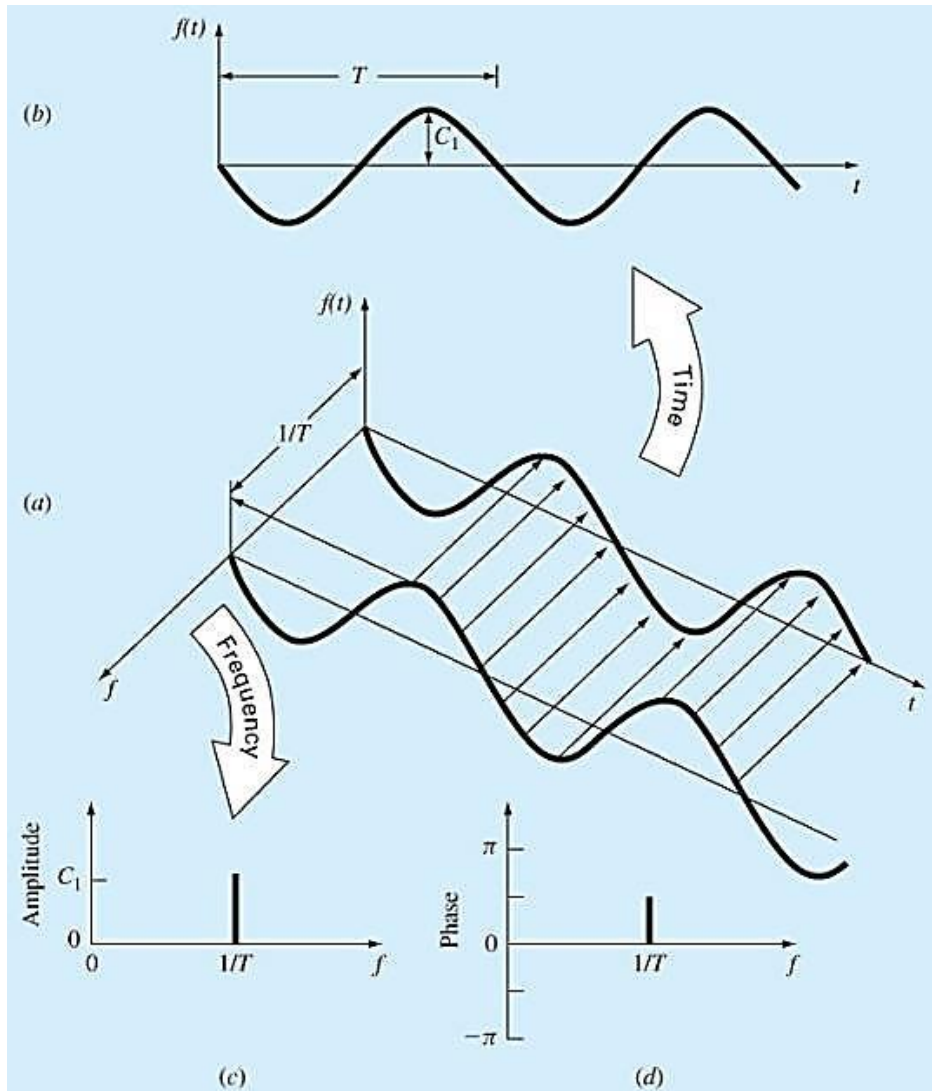
$$f(t) = A_0 + A_1 \cos(\omega_0 t) + B_1 \sin(\omega_0 t)$$



The two forms are related by

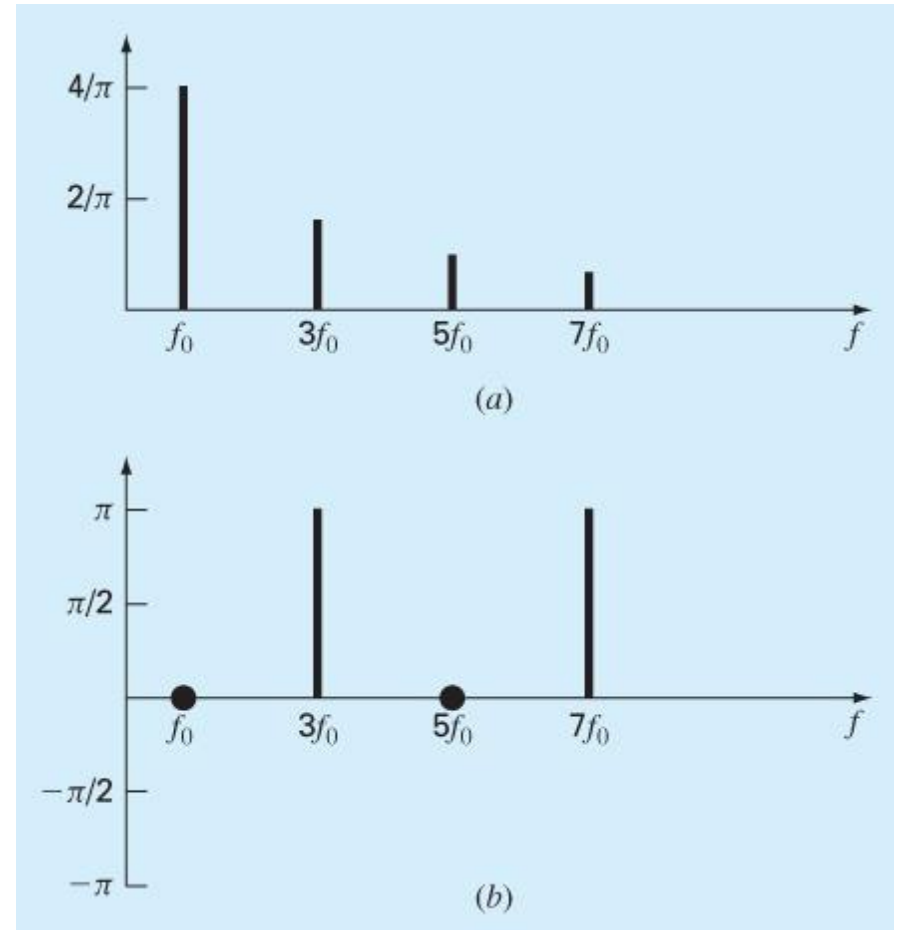
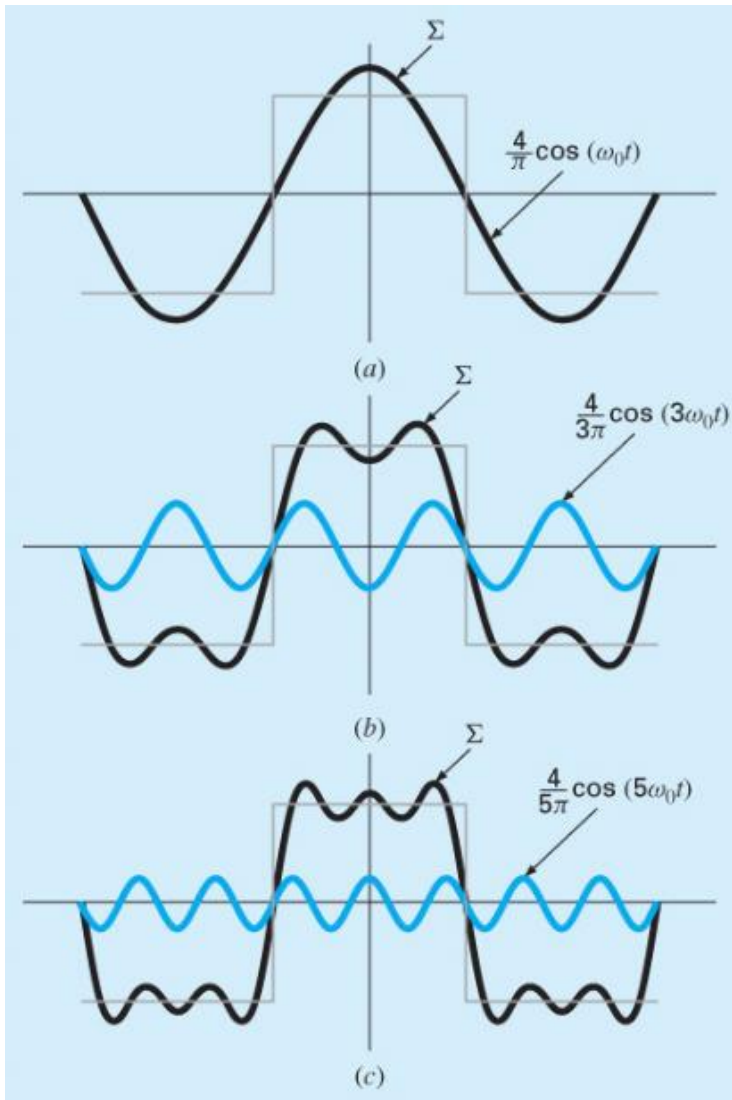
$$C_1 = \sqrt{A_1^2 + B_1^2} \qquad \theta = \arctan(-B_1/A_1)$$

Time vs. Frequency Domains



Frequency domain provides an alternative perspective for characterizing the behavior of oscillating functions

Fourier Series approx. and transform



Fourier Series and Fourier Transform

Continuous Fourier Series:

$$f(t) = a_0 + \sum_{k=1}^{\infty} [a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t)]$$

Coefficients:

$$a_k = \frac{2}{T} \int_0^T f(t) \cos(k\omega_0 t) dt$$

$$b_k = \frac{2}{T} \int_0^T f(t) \sin(k\omega_0 t) dt$$

Compact form:

$$f(t) = \sum_{k=-\infty}^{\infty} \tilde{c}_k e^{ik\omega_0 t}$$

Coefficients:

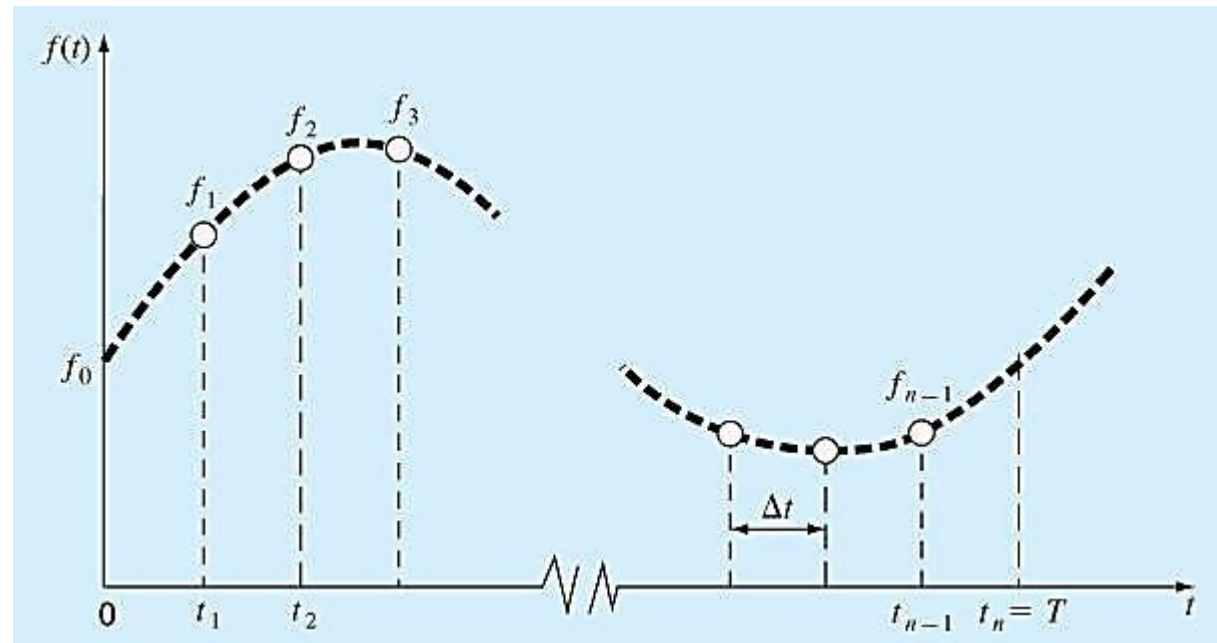
$$\tilde{c}_k = \frac{1}{T} \int_{-T/2}^{T/2} f(t) e^{-ik\omega_0 t} dt$$

Discrete Fourier Transform (DFT)

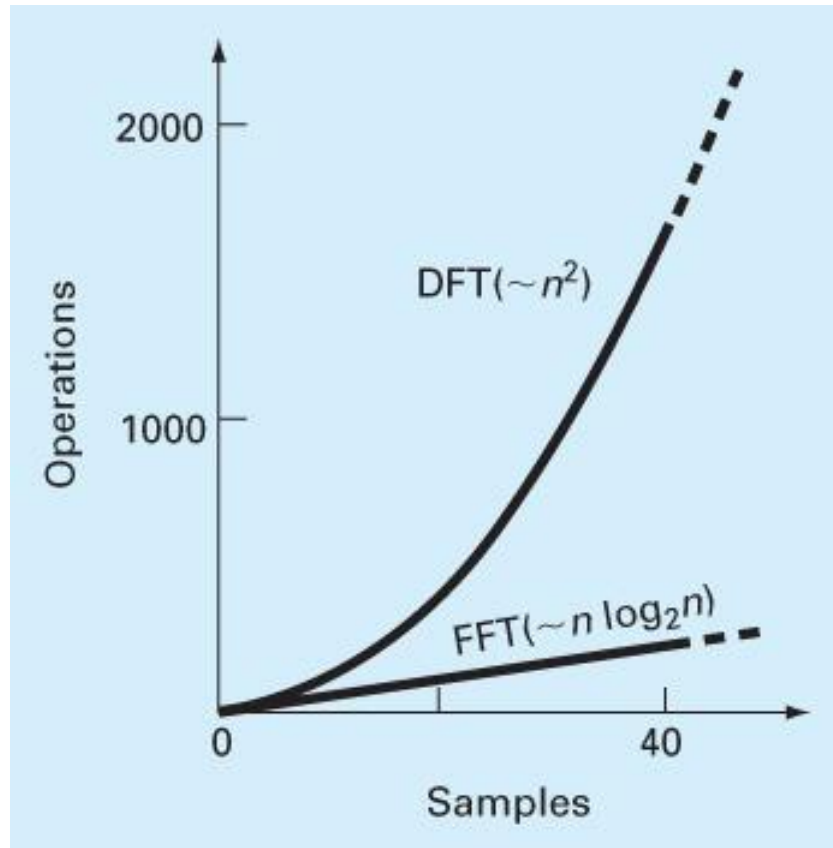
In engineering, data are often collected in or converted to discrete format.

Discrete Fourier transform: $F_k = \sum_{j=0}^{n-1} f_j e^{-ik\omega_0 j}$ for $k = 0$ to $n-1$

$$\omega_0 = \frac{2\pi}{n}$$



MATLAB built-in function: **fft**



DFT is computationally burdensome.

A Fast Fourier Transform (FFT) algorithm has been developed to compute the DFT in an economical fashion.

MATLAB has a built-in function to compute FFT

$$\mathbf{F} = \text{fft}(\mathbf{f}, n)$$

$\mathbf{F}$ = a vector containing the DFT

$\mathbf{f}$ = a vector containing the signal/ discrete measurements

Example Problem: DFT

Apply MATLAB's fft function to determine the DFT for the given data:

t	0	0.02	0.04	0.06	0.08	0.1	0.12	0.14
f(t)	6.0	5.7	3.0	5.7	6.0	4.3	5.0	4.3

Solution:

Sampling frequency = $(1/0.02\text{s}) = 50\text{Hz}$

The Nyquist frequency = $0.5 \times 50 = 25\text{Hz}$

(The highest frequency that can be measured in a signal is called the **Nyquist frequency** and is half the sampling frequency)

Lowest detectable frequency = $1/(n \times \Delta t) = 1/0.16 = 6.25\text{ Hz}$

Therefore, the analysis should be able to detect signals with periods from $1/25 = 0.04$ upto $1/6.25 = 0.16\text{s}$.

Example Problem: DFT

```
Y=fft(y)/n;
```

```
**
```

```
Y.'
```

show numbers
in columnwise pattern serially

ans =

5.0000	+	0.0000i
0.0000	+	0.0000i
0.5000	+	0.0000i
-0.0000	-	0.5000i
0.0000	+	0.0000i
-0.0000	+	0.5000i
0.5000	+	0.0000i
0.0000	-	0.0000i

$\cos \theta$

$\sin \theta$

Example Problem: DFT

Apply MATLAB's fft function to determine the DFT for a simple sinusoid:

$$f(t) = 5 + \cos(2\pi(12.5)t) + \sin(2\pi(18.75)t)$$

Generate 8 equispaced points with $\Delta t = 0.02$ s. Plot the results versus frequency

Solution: $\overset{\frac{n}{f_s}}{\text{Total sample length}} = 8/50 = 0.16 \text{ s}$ or, $\overset{n \cdot \Delta t}{8 \times 0.02 = 0.16 \text{ s}}$

Sampling frequency = $(1/0.02\text{s}) = 50\text{Hz}$ (f_s)

The **Nyquist frequency** = $0.5 \times 50 = 25\text{Hz}$

(The **highest frequency** that can be **measured** in a **signal** is called the **Nyquist frequency** and is half the sampling frequency) (Assumed)

$\text{Total sample length} = 8/50 = 0.16 \text{ s}$ (no. of equispaced points / sampling frequency)

Lowest detectable frequency = $1/(n \cdot \Delta t) = 1/0.16 = 6.25\text{Hz}$ (Mukhoto formula)

Therefore, the analysis should be able to detect signals with periods from $1/25 = 0.04$ upto $1/6.25 = 0.16\text{s}$.

Example Problem: DFT

```

clc; clear all; close all;
n=8; dt=0.02; fs=1/dt; % (sampling frequency)
t_k = k * dt = k * 1/fs [∴ dt = 1/fs]
(mukhosto) tspan=(0:n-1)/fs; Alt: 0:0.02:0.14 or, linspace(0, 0.14, 8)
y=5+cos(2*pi*12.5*tspan)+sin(2*pi*18.75*tspan);
subplot(3,1,1);
plot(tspan,y,'-
ok','linewidth',2,'MarkerFaceColor','black');
title('(a) f(t) versus time (s)');
%%%%%%%%%%
Y=fft(y)/n;
Y.'

```

Code move rakhan mukhosto part:

```

n = 8; t = 0:0.02; fs = 1/dt;
nyquist = fs/2; fmin = 1/(n * dt); f = linspace(fmin, nyquist, n/2);
tspan = (0:n-1)/fs;
y =
Y = fft(y)/n; Y.'
Y(1) = [1; YP = Y(1:n/2);
Pyy = abs(YP).^2;

```

ans =

```

5.0000 + 0.0000i
0.0000 + 0.0000i
0.5000 + 0.0000i
-0.0000 - 0.5000i
0.0000 + 0.0000i
-0.0000 + 0.5000i
0.5000 + 0.0000i
0.0000 - 0.0000i

```

Example Problem: DFT

`figure(1)` *(mukhosto) → (thaka na thaka matter kortese na)*
`nyquist=fs/2; fmin=1/(n*dt);` *(formula)*
`f=linspace(fmin,nyquist,n/2);` *(4 to equispaced points)*
`Y(1)=[];` **** Removes 1st Value* *why??*
`YP=Y(1:n/2);`

$$Y = \begin{bmatrix} Y(1) & Y(2) & Y(3) & \dots & Y(n) \end{bmatrix}$$

$$YP = \begin{bmatrix} Y(1) & Y(2) & Y(3) & Y(4) \end{bmatrix}$$
but why?? (myb process ke jone)

After executing `Y(1)=[]`

Y =

```

0.0000 + 0.0000i
0.5000 + 0.0000i
-0.0000 - 0.5000i
0.0000 + 0.0000i
-0.0000 + 0.5000i
0.5000 + 0.0000i
0.0000 - 0.0000i

```

figure(1):

- Selects figure window #1
- Creates it if necessary
- Makes it the active plotting target
- Ensures subsequent plotting commands go into that window

stem():

- Plots discrete data
- Draws vertical lines from zero to each data point
- Emphasizes that FFT output consists of discrete frequency bins
- Is the correct visualization for DFT/FFT results

```

subplot(3,1,2);
stem(f,real(YP),'linewidth',2,'MarkerFaceColor','blue');
grid; title('(b) Real component versus frequency')

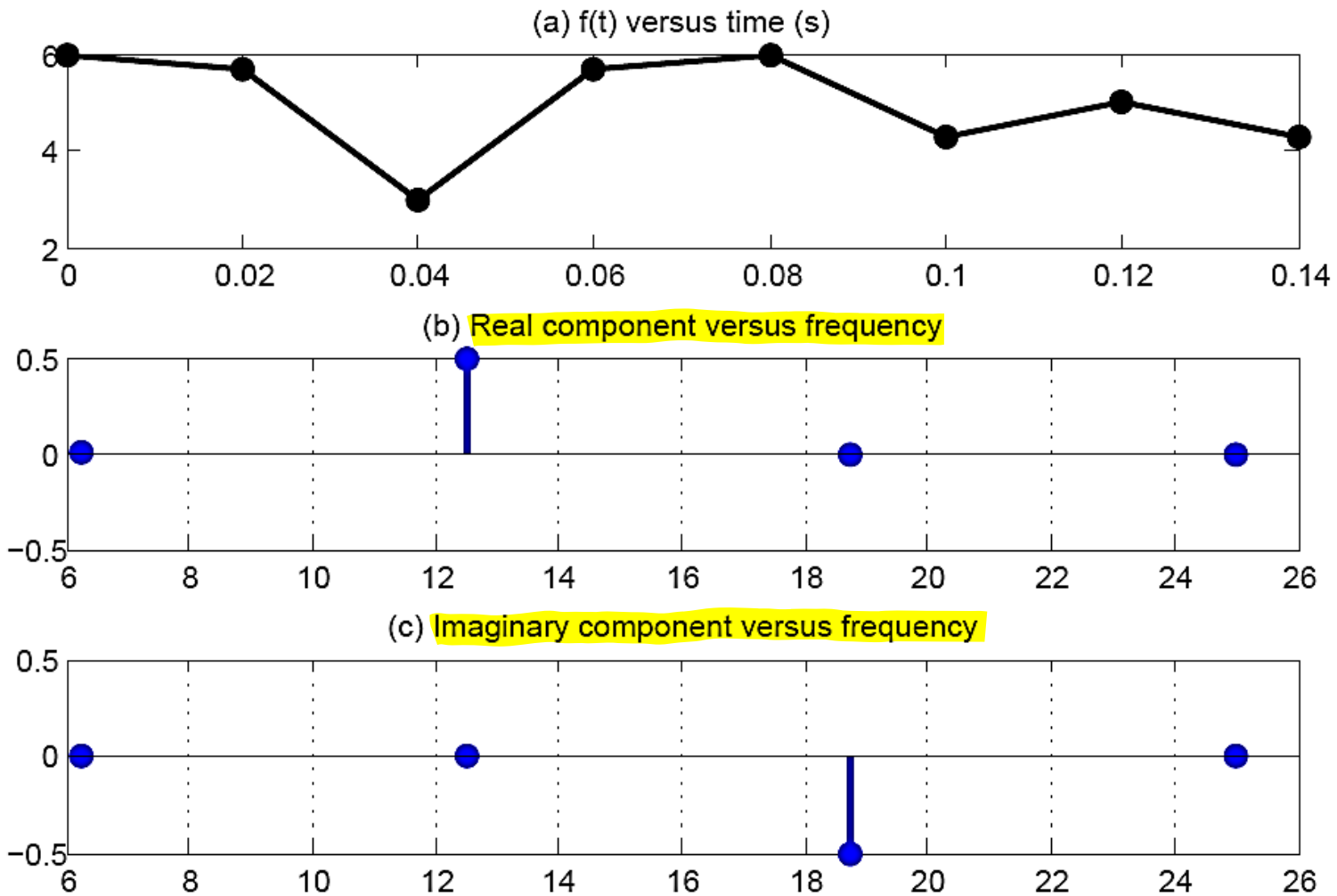
```

```

subplot(3,1,3);
stem(f,imag(YP),'linewidth',2,'MarkerFaceColor','blue');
grid; title('(c) Imaginary component versus frequency')

```

Example Problem: DFT



Power Spectrum

Provides another useful way to **discern** the **underlying harmonics** of **seemingly random signals**.

In terms of DFT, a **power spectrum** consists of a **plot of power** associated with **each frequency component** (P_k) versus **frequency**.

Power = sum of square of Fourier coefficients

$$P_k = |\tilde{c}_k|^2$$

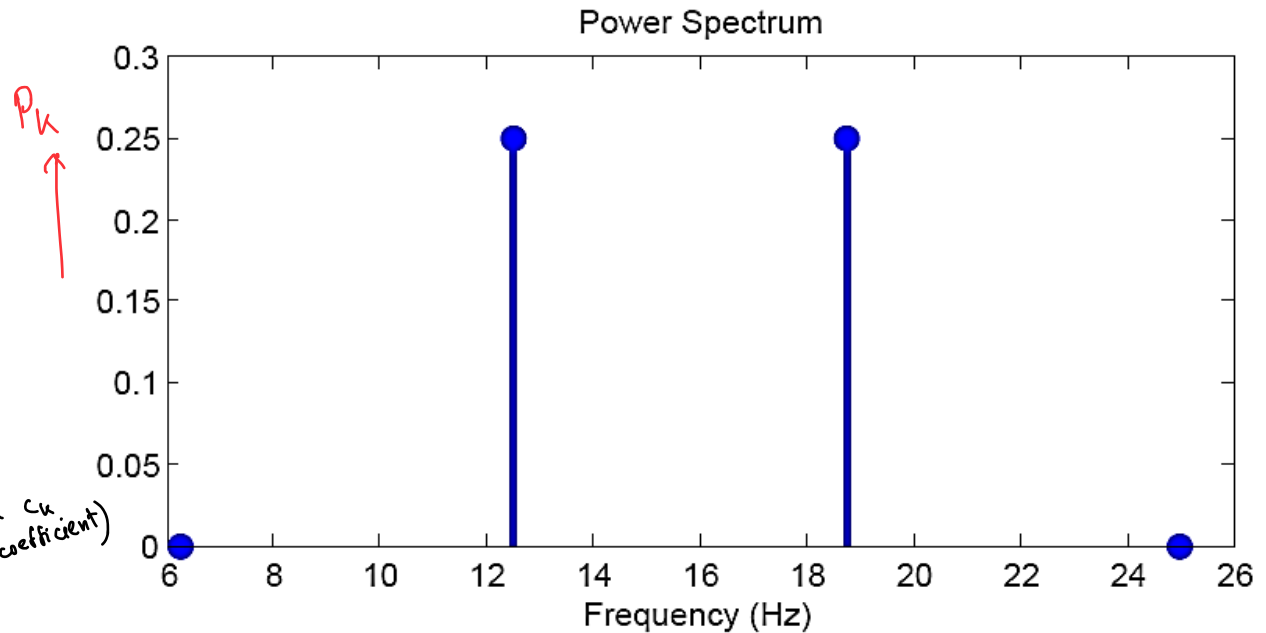
$$P_k = \sum (c_k)^2$$

figure (2)

```
Pyy=abs(YP).^2;
```

```
stem(f,Pyy,'linewidth',2,'MarkerFaceColor','blue');
```

```
title('Power Spectrum'); xlabel('Frequency (Hz)');
```



(A physical time series used as an example to demonstrate spectral decomposition.)

Example Problem: Wolf Sunspot Number

The data for year-wise sunspot number are contained in the MATLAB file **sunspot.dat**. Perform Fourier analysis on the time series data and determine the cycle length. Compare it to J.R. Wolf's estimate of cycle length (~11.1 years).

```
clear all; close all;  
load sunspot.dat  
year=sunspot(:,1);  
number=sunspot(:,2);
```

(inbuilt matlab m-file)
1st column → year
2nd column → sunspot number

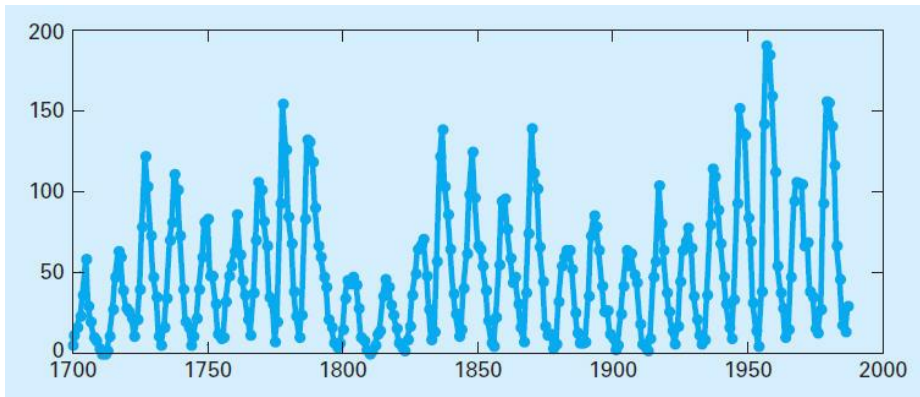
```
n=length(number);  
a=polyfit(year, number,1);  
lineartrend=polyval(a,year);  
ft=number-lineartrend;
```

(residual)
(predicted numbers)
(multithro)

```
F=fft(ft);  
fs=1; f=(0:n/2)*fs/n;  
pow=abs(F(1:n/2+1)).^2;
```

'residual en fft' but why?

[same dimension]



Removing the upward linear trend of the data

$$\frac{n}{2} \times \frac{f_s}{n} = \frac{f_s}{2}$$

nyquist frequency (max.)

Example Problem: Wolf Sunspot Number

```
plot(f,pow)
xlabel('Frequency (cycles/year)'); ylabel('power')
title('Power versus frequency')
```

